

State simulator dfference between Cirq and TFQ

<https://github.com/tensorflow/quantum/issues/235>

ROW 29

State simulator dfference between Cirq and TFQ #235

[New issue](#)[Closed](#)

#236



therooler opened on May 7, 2020

...

Environment:

Python 3.7

tensorflow = 2.1.0

tensorflow-quantum = 0.3.0

Cirq = 0.7.0

Issue: When running larger ($N = 12$) variational circuits `cirq.Simulator()` and `tfq.layers.State` output states that do not agree. When using this in VQE to find the ground state of some Hamiltonian, after convergence, the Cirq state has overlap ~ 1 with the ground state whereas the TFQ state does not. This problem gets worse as N increases.

Reproducible example

We run a depth 4 circuit with ZZ gates between nearest neighbors on an open chain of length N .

With $N=8$, the overlap between the two states is 1. But when we go to a chain of length 12, this is no longer the case.

```
import cirq
import tensorflow_quantum as tfq
import tensorflow as tf
import numpy as np
import sympy

def unique_name():
    """
    Generator to generator an infinite number of unique names

    Yields: Strings of the form 'theta_<number>'

    """
    num = 1
    while True:
        yield 'theta_' + str(num)
        num += 1

def issue(nqubits):
    depth = 4

    name_generator = unique_name()

    qubits = cirq.GridQubit.rect(nqubits, 1)
    circuit = cirq.Circuit()

    control_params = []

    circuit.append(cirq.H.on_each(qubits))
    for _ in range(depth):
        name = next(name_generator)
        control_params.append(name)
        symbol = sympy.Symbol(name)

        circuit.append(cirq.ZZ(qubits[i], qubits[i + 1]) ** symbol for i in
                        range(0, nqubits - 1, 2))
        circuit.append(cirq.ZZ(qubits[i], qubits[i + 1]) ** symbol for i in
                        range(1, nqubits - 1, 2))
        circuit.append(cirq.ZZ(qubits[nqubits - 1], qubits[0]) ** symbol)

    initial_value = np.random.rand(1, len(control_params))

    params_dict = dict(zip(control_params, initial_value[0]))
    resolver = cirq.ParamResolver(params_dict)
    phi_cirq = cirq.Simulator().simulate(circuit, resolver).final_state

    state_layer = tfq.layers.State()
    model_params = tf.Variable(initial_value=initial_value)
    # phi_tf = \
    #     state_layer(circuit, symbol_names=control_params,
    #                 symbol_values=initial_value)[
```

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

[Precision fix](#)

tensorflow/quantum

Notifications

[Customize](#) **Subscribe**

You're not receiving notifications from this thread.

Participants

[Visual](#)

```

params_dict = dict(zip(control_params, initial_value[0]))
resolver = cirq.ParamResolver(params_dict)
phi_cirq = cirq.Simulator().simulate(circuit, resolver).final_state

state_layer = tfq.layers.State()
model_params = tf.Variable(initial_value=initial_value)
# phi_tfq = \
#     state_layer(circuit, symbol_names=control_params,
#                 symbol_values=initial_value)[
#                 0].numpy()
phi_tfq = \
    state_layer(circuit, symbol_names=control_params,
                symbol_values=model_params)[
                0].numpy()

overlap = np.abs(np.conj(phi_cirq) @ phi_tfq)
return overlap

if __name__ == "__main__":
    print(issue(8))
    print(issue(12))

>>> 0.9999997
>>> 0.5101427

```

Is our implementation incorrect? Or have we encountered a bug?



MichaelBroughton on May 7, 2020

Collaborator ...

Was able to recreate on my end, tried using `backend=cirq.Simulator()` as well, which did bring things back to 0.99. My plan now is to review all of the changes that might have been made between our C++ version of `qsim` and the reference version <https://github.com/quantumlib/qsim> . Will report back with findings soon.



MichaelBroughton mentioned this on May 8, 2020

[Precision fix #236](#)



MichaelBroughton closed this as **completed** in [#236](#) on May 9, 2020



jaeyoo added a commit that references this issue on Mar 30, 2023

Merge pull request [tensorflow#235](#) from quantumlib/thread-recommend ...

Verified 4705b09

Precision fix #236

<> Code

Merged MichaelBroughton merged 3 commits into master from precision_fix on May 9, 2020

Conversation 6 Commits 3 Checks 0 Files changed 4 +132 -24

MichaelBroughton commented on May 8, 2020 Collaborator

Fixes #235

Reviewers

- zaqqwerty ✓
- philiptmassey

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

State simulator difference between Cirq and ...

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

2 participants

zaqqwerty MichaelBroughton

MichaelBroughton added 2 commits 5 years ago

- Higher qubit precision fix. d5b4584
- missed format. 7834eaf

MichaelBroughton requested review from philiptmassey and zaqqwerty 5 years ago

zaqqwerty suggested changes on May 8, 2020 View reviewed changes

zaqqwerty left a comment Contributor

Couple of small changes

```
tensorflow_quantum/core/ops/circuit_execution_ops_test.py
238 + symbol_names = []
239 + circuit_batch, resolver_batch = \
240 + util.random_circuit_resolver_batch(
241 + cirq.GridQubit.rect(4, 4), 5)
```

zaqqwerty on May 8, 2020 Contributor

I think we also need to test the specific case from the original bug, which used a line of 12 qubits

MichaelBroughton on May 9, 2020 Collaborator Author

The case of (4,4) actually proved to be worse than the (1,12) case in my testing.

Reply...

```
tensorflow_quantum/core/src/program_resolution_test.cc
```

Show resolved

New changes since you last viewed View changes

A feedback. 8927c66

MichaelBroughton requested a review from zaqqwerty 5 years ago